

Antoine

Lecomte

Lab 5 : Supply-chain security

Question 1 :

On identifie les Pods déployés par Helm Chart en recherchant les noms de contrôleurs. Chercher « kind: » pour retrouver tous leurs noms.

On peut trouver :

NetworkPolicy, PodDisruptionBudget, ServiceAccount, Secret, ConfigMap, Role, RoleBinding, Service, Deployment, StatefulSet, PersistentVolumeClaim.

Les Pods sont créés par les contrôleurs suivants :

Deployment : test-spring-cloud-dataflow-server, test-spring-cloud-dataflow-skipper

StatefulSet : test-mariadb, test-rabbitmq

Les autres contrôleurs ne créent pas directement de Pods, ils sont utilisés pour leur configuration, sécurisation et exposition ainsi que pour les applications.

Exemple :

```
# Source: spring-cloud-dataflow/templates/server/deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: test-spring-cloud-dataflow-server
  labels:
    app.kubernetes.io/instance: test
    app.kubernetes.io/managed-by: Helm
    app.kubernetes.io/name: spring-cloud-dataflow
    app.kubernetes.io/version: 2.11.5
    helm.sh/chart: spring-cloud-dataflow-34.2.1
    app.kubernetes.io/component: server
  namespace: "default"
spec:
```

Question 2 :

Chaque Pod contient un seul conteneur principal avec éventuellement des initContainers.

Pour le Pod test-spring-cloud-dataflow-server : 1 conteneur server.

Pour le Pod test-spring-cloud-dataflow-skipper : 1 conteneur skipper.

Pour le Pod test-mariadb : 1 conteneur mariadb.

Pour le Pod test-rabbitmq : 1 conteneur rabbitmq.

On ne liste pas les initContainers, il y a donc 4 conteneurs au total.

Leurs noms sont définis dans la section « spec.template.spec.containers » pour chaque Deployment ou StatefulSet.

```
containers:
  - name: server
    image: docker.io/bitnami/spring-cloud-dataflow:2.11.5-debian-12-r9
    imagePullPolicy: "IfNotPresent"
    securityContext:
```

Question 3 :

Pod : test-mariadb / Services : test-mariadb-headless, test-mariadb / Type : ClusterIP.

Pod : test-rabbitmq / Services : test_rabbitmq-headless, test-rabbitmq / Type : ClusterIP.

Pod : test-spring-cloud-dataflow-server / Service : test-spring-cloud-dataflow-server / Type : ClusterIP.

Pod : test-spring-cloud-dataflow-skipper / Service : test-spring-cloud-dataflow-skipper / Type : ClusterIP.

On peut voir les sélecteurs pour cibler les Pods dans la section « spec.selector ».

Tous les services ici sont de type ClusterIP, car ils ne sont accessibles qu'à l'intérieur du cluster Kubernetes (ClusterIP par défaut).

```
spec:
  replicas: 1
  selector:
    matchLabels:
      app.kubernetes.io/instance: test
      app.kubernetes.io/name: spring-cloud-dataflow
      app.kubernetes.io/component: server
```

Question 4 :

Service Account : test-mariadb / Pod associé : test-mariadb (conteneur StatefulSet).

Service Account : test-rabbitmq / Pod associé : test-rabbitmq (conteneur StatefulSet).

Service Account : test-spring-cloud-dataflow / Pods associés : test-spring-cloud-dataflow-server (conteneur Deployment) et test-spring-cloud-dataflow-skipper (conteneur Deployment).

3 service Accounts sont définis.

Voire section « spec.template.spec.serviceAccountName » :

```
spec:
  serviceAccountName: test-spring-cloud-dataflow
  automountServiceAccountToken: true
  ...
```

Question 5 :

2 rôles définis dans la configuration Kubernetes, chercher « kind: Role ». Leur nom est en section « metadata.name », les permissions dans « rules ».

Pour le rôle test-rabbitmq-endpoint-reader :

```
# Source: spring-cloud-dataflow/charts/rabbitmq/templates/role.yaml
kind: Role
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: test-rabbitmq-endpoint-reader
  namespace: "default"
  labels:
    app.kubernetes.io/instance: test
    app.kubernetes.io/managed-by: Helm
    app.kubernetes.io/name: rabbitmq
    app.kubernetes.io/version: 4.0.7
    helm.sh/chart: rabbitmq-15.3.3
rules:
- apiGroups: [""]
  resources: ["endpoints"]
  verbs: ["get"]
- apiGroups: [""]
  resources: ["events"]
  verbs: ["create"]
---
```

Pour le rôle test-spring-cloud-dataflow :

```
# Source: spring-cloud-dataflow/templates/role.yaml
kind: Role
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: test-spring-cloud-dataflow
  # ...
rules:
- apiGroups: [""]
  resources: ["services", "pods", "replicationcontrollers", "persistentvolumeclaims"]
  verbs: ["get", "list", "watch", "create", "delete", "update"]
- apiGroups: [""]
  resources: ["configmaps", "secrets", "pods/log", "pods/status"]
  verbs: ["get", "list", "watch"]
- apiGroups: ["apps"]
  resources: ["statefulsets", "deployments", "replicasets"]
  verbs: ["get", "list", "watch", "create", "delete", "update", "patch"]
- apiGroups: ["extensions"]
  resources: ["deployments", "replicasets"]
  verbs: ["get", "list", "watch", "create", "delete", "update", "patch"]
- apiGroups: ["batch"]
  resources: ["cronjobs", "jobs"]
  verbs: ["get", "list", "watch", "create", "delete", "update", "patch"]
```

Question 6 :

2 rôles associés définis dans la configuration Kubernetes, chercher « kind: RoleBinding ». Leur nom est en section « metadata.name », les Service Accounts associés dans « subjects.name » et les rôles liés dans « roleRef.name ».

Pour le rôle associé test-rabbitmq-endpoint-reader :

```
# Source: spring-cloud-dataflow/charts/rabbitmq/templates/rolebinding.yaml
kind: RoleBinding
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: test-rabbitmq-endpoint-reader
  namespace: "default"
  labels:
    app.kubernetes.io/instance: test
    app.kubernetes.io/managed-by: Helm
    app.kubernetes.io/name: rabbitmq
    app.kubernetes.io/version: 4.0.7
    helm.sh/chart: rabbitmq-15.3.3
subjects:
- kind: ServiceAccount
  name: test-rabbitmq
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: Role
  name: test-rabbitmq-endpoint-reader
```

Pour le rôle associé test-spring-cloud-dataflow :

```
# Source: spring-cloud-dataflow/templates/rolebinding.yaml
kind: RoleBinding
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: test-spring-cloud-dataflow
  labels:
    app.kubernetes.io/instance: test
    app.kubernetes.io/managed-by: Helm
    app.kubernetes.io/name: spring-cloud-dataflow
    app.kubernetes.io/version: 2.11.5
    helm.sh/chart: spring-cloud-dataflow-34.2.1
  namespace: "default"
roleRef:
  kind: Role
  name: test-spring-cloud-dataflow
  apiGroup: rbac.authorization.k8s.io
subjects:
- kind: ServiceAccount
  name: test-spring-cloud-dataflow
  namespace: "default"
```

Question 7 :

Chercher « kind: NetworkPolicy », le nom de chacune se trouve dans la section « metadata.name ».

NetworkPolicy de test-mariadb :

Les Pods avec les labels suivants (section « spec.podSelector.matchLabels ») sont concernés :

```
Labels:
  app.kubernetes.io/instance: test
  app.kubernetes.io/managed-by: Helm
  app.kubernetes.io/name: mariadb
  app.kubernetes.io/version: 11.4.5
  helm.sh/chart: mariadb-20.4.1
  app.kubernetes.io/part-of: mariadb
```

Port autorisé en connexion entrante : 3306 (MySQL).

Ports autorisés en connexion sortante : toutes ({}).

Illustration :

```
policyTypes:
- Ingress
- Egress
egress:
- {}
ingress:
- ports:
  - port: 3306
  - port: 3306
```

NetworkPolicy de test-rabbitmq :

Les Pods avec les labels app.kubernetes.io/instance: test et app.kubernetes.io/name: rabbitmq, sont concernés.

Ports autorisés en connexion entrante : 4369 (epmd), 5672 (AMQP), 5671 (AMQP/TLS), 25672 (inter-node), 15672 (HTTP API).

Ports autorisés en connexion sortante : toutes ({}).

NetworkPolicy de test-spring-cloud-dataflow-server :

Les pods avec les labels app.kubernetes.io/instance: test, app.kubernetes.io/name: spring-cloud-dataflow et app.kubernetes.io/component: server, sont concernés.

Port autorisé en connexion entrante : 8080 (HTTP).

Ports autorisés en connexion sortante : toutes ({}).

NetworkPolicy de test-spring-cloud-dataflow-skipper :

Les pods avec les labels app.kubernetes.io/instance: test, app.kubernetes.io/name: spring-cloud-dataflow et app.kubernetes.io/component: skipper, sont concernés.

Ports autorisés en connexion entrante : 7577 (HTTP), 80 (HTTP).

Ports autorisés en connexion sortante : toutes ({}).

Question 8a :

Chercher « kind: Secret ». 3 contenus secrets en tout. Le nom dans la section « metadata.name ».

Secret test-mariadb utilise un mot de passe root pour MariaDB encodé en base64 et un mot de passe utilisateur également encodé (voire section « data ») :

```
data:
  mariadb-root-password: "NE82cjJ6Q01Vbw=="
  mariadb-password: "Y2hhbmdlLW1l"
```

Secret test-rabbitmq-config utilise un fichier (rabbitmq.conf) de configuration pour RabbitMQ encodé en base64, il contient des paramètres dissimulés (des ports, utilisateurs, etc.).

Secret test-rabbitmq contient le mot de passe pour l'utilisateur RabbitMQ (toujours encodé en base64) : rabbitmq-password, ainsi qu'un cookie Erlang pour la communication entre nœuds RabbitMQ : rabbit-erlang-cookie.

Question 8b :

Report Summary

Target	Type	Vulnerabilities	Secrets
rabbitmq:3.13 (ubuntu 24.04)	ubuntu	12	-

Legend:

- '-': Not scanned
- '0': Clean (no security findings detected)

rabbitmq:3.13 (ubuntu 24.04)

Total: 12 (UNKNOWN: 0, LOW: 7, MEDIUM: 5, HIGH: 0, CRITICAL: 0)

Aucune vulnérabilité de gravité élevée ou critique rencontrée.

libpam-modules	CVE-2025-8941	MEDIUM	1.5.3-5ubuntu5.5	linux-pam: Incomplete fix for CVE-2025-6020 https://avd.aquasec.com/nvd/cve-2025-8941
----------------	---------------	--------	------------------	--

Rapport de sécurité pour la CVE-2025-8941 :

- obtient une vulnérabilité moyenne
- package impacté : libpam-modules
- version affectée : 1.5.3-5ubuntu5.5
- composant : Linux-PAM (Pluggable Authentication Modules)

C'est une bibliothèque utilisée par la majorité des distributions Linux pour la gestion de l'authentification. Elle est assez sensible car elle touche le mécanisme central d'authentification du système.

La version ici CVE-2025-8941 est une correction incomplète d'une vulnérabilité précédente, CVE-2025-6020, qui avait été corrigée, mais le correctif est en fait incomplet, ce qui laisse une partie de la faille toujours exploitable.

La vulnérabilité est un problème dans la gestion des modules PAM, qui permet dans certains scénarios de contourner partiellement les protections mises en place.

Question 9 :

- a) Non, runAsNonRoot : true, donc exécution interdite avec UID 0, et runAsUser : 1001, force l'usage d'un utilisateur autre que root.
- b) Non, allowPrivilegeEscalation: false.
- c) Aucune, capabilities : drop : -ALL. Le conteneur fonctionne avec l'ensemble minimal de permissions.
- d) Non, readOnlyRootFilesystem : true. Le container peut seulement écrire dans /tmp, mais pas dans /, /var, /usr, ...
- e) Non, pas de profil explicite AppArmor dans les sections (comme apparmor.security.beta.kubernetes.io/... par exemple).
- f) Oui, seccompProfile : type : RuntimeDefault. Il bloque les appels systèmes dangereux (comme ptrace, kexec, unshare, mount, etc.) et permet uniquement un ensemble sûr d'appels système.

Question 10 :

Les services test-mariadb-headless (4 ports) et test-mariadb (avec MySQL : 3306) n'ont pas d'exposition externe au cluster Kubernetes, uniquement dans le cluster donc l'exposition est minimale.

Pour test-rabbitmq-headless et test-rabbitmq, de même pour les 5 ports exposés.

Pour test-spring-cloud-dataflow-server, 8080 (HTTP) est utilisable depuis le cluster, si l'on avait besoin de l'application depuis l'extérieur il faudrait un Ingress ou LoadBalancer configuré, mais ce n'est pas le cas. C'est une exposition minimale.

Et pour test-spring-cloud-dataflow-skipper, c'est aussi minimal pour le port 80.

Tous sont de type ClusterIP, l'exposition aux attaques externes est limitée et les ports sont utilisés pour les besoins de l'application donc tous les ports ouverts ont une nécessité. Un port non-utilisé peut justement être retiré pour réduire la surface d'attaque interne.

Question 11 :

test-spring-cloud-dataflow accorde les permissions les plus étendues dans la configuration. Ce rôle est associé aux Pods Deployment (les composants server et skipper).

Il permet des opérations create, read, update, delete sur plusieurs ressources critiques pods, services, deployments, secrets. Il permet le patch (modifications partielles des ressources) et couvre plusieurs groupes d'API (« », « apps », « extensions », « batch »).

Question 12 :

Pour test-spring-cloud-dataflow-server, l'accès entrant est autorisé avec le port 8080 (HTTP) comme vu précédemment. Tous les Pods du cluster peuvent accéder à ce port 8080 de ces Pods sauf en cas de restriction supplémentaire (venant d'une autre NetworkPolicy, qui viendrait bloquer le trafic). En accès sortant, la communication est possible avec n'importe quel autre Pod ou service à l'intérieur ou extérieur du cluster.

Pour test-spring-cloud-dataflow-skipper, les ports autorisés en accès entrant sont donc 7577 et 80 (HTTP) et comme pour le server, tous les Pods du cluster peuvent accéder aux ports 7577 et 80 de ces Pods, sauf restriction supplémentaire. En accès sortant, la communication est possible avec n'importe quel autre Pod ou service à l'intérieur ou extérieur du cluster.

Il n'y a pas de restriction explicites des networkpolicies actuelles concernant l'accès entrant à des Pods spécifiques (ex : un « from: [...] » dans Ingress), donc n'importe quel Pod dans le cluster peut communiquer avec ces Pods sur les ports spécifiés. Avec cette règle, on peut spécifier des Pods et namespaces que l'on souhaite autoriser explicitement à se connecter pour encore une fois réduire la surface d'attaque.

Question 13 :

En partie oui, car les valeurs des Secrets des mots de passe et cookies sont encodées en base64 directement dans les fichiers YAML du Chart mais ces valeurs sont visibles en clair bien qu'elles soient encodées, donc elles peuvent être facilement décodées. Un intrus ayant accès au dépôt ou aux fichiers du Chart peut donc récupérer les secrets.

Les Secrets sont créés via les ressources Kubernetes Secret dans les fichiers YAML du Chart puis sont montés en tant que volumes ou variables d'environnement dans les Pods. Le fait que les valeurs soient hardcodées dans le Chart n'est pas sécurisé.

On peut améliorer la sécurité car les valeurs sont visibles dans les fichiers YAML et sont hardcodées, car Base64 n'est pas du chiffrement mais juste un encodage et qu'on pourrait tout à fait chiffrer les messages, mais encore en utilisant des mots de passe suivant une rotation automatique pour les Secrets. On peut aussi limiter plus fortement l'accès aux Secrets dans les Network Policies et mettre en place des audits réguliers.

Question 14 :

On peut décrire un scénario d'attaque plausible basé sur les configurations et le framework MITRE ATT&CK pour Kubernetes avec les vulnérabilités trouvées :

- 1- Exploiter une vulnérabilité dans un conteneur comme une version non patchée de Spring Cloud DataFlow ou RabbitMQ. L'attaquant obtient un shell dans un Pod (exemple : dans test-spring-cloud-dataflow-server).

- 2- Abus d'une configuration de sécurité. On peut s'exécuter en tant qu'utilisateur 1001 (non-root) mais aucune restriction supplémentaire comme `readOnlyRootFilesystem: true` ou `allowPrivilegeEscalation: false`, n'est appliquée sur tous les conteneurs. Si l'attaquant trouve une faille dans l'application, il peut élever ses privilèges dans le Pod qu'il a maintenant accédé.
- 3- Accès aux Secrets hardcodés. L'attaquant fait juste un « `ls /etc/secrets` » dans le Pod maintenant qu'il est root. Il trouve les secrets hardcodés, décode les valeurs base64 et récupère les mots de passe pour MariaDB et RabbitMQ. Il a un accès complet aux ressources de MariaDB et peut les dérober ou les modifier. Il peut intercepter les messages de RabbitMQ.
- 4- Liste des ressources Kubernetes. Grâce au rôle test-spring-cloud-dataflow très permissif (vu question 11), l'attaquant liste tous les pods, deployments, services, secrets et configmaps dans le namespace default : il peut faire « `kubectl get pods, deployments, secrets --all-namespaces` » et a accès à de nouvelles cibles (exemple : test-rabbitmq, test-mariadb).
- 5- Abus des permissions réseau. Les Network Policies actuelles autorisent tout le trafic entrant sur les ports 8080 (Dataflow), 7577/80 (Skipper), 3306 (MariaDB) et autres ports. Donc il se connecte depuis le Pod compromis vers RabbitMQ sur le port 5672, envoie ce qu'il veut dessus. Il se connecte sur MariaDB sur le port 3306 et peut totalement modifier les données (comme il n'y a aucune restriction « from: » dans les Network Policies, ce qui n'empêche pas l'ensemble des Pods de pouvoir communiquer entre eux).
- 6- Impact final. Dans le scénario de déni de service (DoS) l'attaquant supprime les Pods critiques comme test-rabbitmq ou test-mariadb et PodDisruptionBudgets limitent les dégâts hormis si l'attaquant supprime rapidement plusieurs Pods, le service devient assez inutile car il peut être interrompu. Dans le scénario de Resource Hijacking, l'attaquant envoie un crypto-miner dans un Pod (le Role permissif qu'il a utilisé au départ pour s'infiltrer) et utilise les ressources CPU/mémoire allouées pour l'exemple à Spring Cloud Dataflow pour miner les cryptomonnaies.

Donc l'attaquant pourrait : compromettre un Pod → voler des Secrets → se déplacer latéralement → impacter le cluster (attaque DoS, vol de données ou hijacking). (Pour l'empêcher efficacement, les meilleures défenses en réponse à la question 13.)